

CASE STUDY: USING GANS ON THE CIFAR-10 DATASET

Introduction

Generative Adversarial Networks (GANs) are a class of artificial intelligence algorithms that use two networks, a generator, and a discriminator, to generate data that is similar to some real data. The CIFAR-10 dataset, which contains 60,000 color images in 10 classes, is a popular choice to demonstrate the capability of GANs in generating images.

Problem Description

The CIFAR-10 dataset, while diverse, has a limited number of images per class (6000 images per class). Expanding this dataset with novel but relevant images can be of great benefit for more robust learning. The challenge lies in generating high-quality, diverse, and relevant images that can augment the existing dataset.

Solution - Methodology and Approach

To address the limitation, we can employ GANs to generate new images. The GAN will be trained using the CIFAR-10 dataset. The generator will try to produce images that look like they belong to the CIFAR-10 dataset, while the discriminator will aim to distinguish between real CIFAR-10 images and the fake ones produced by the generator.

GAN Implementation Code

Step 1 : This code imports necessary libraries and modules. It's like preparing all the tools and ingredients needed to cook a dish. We have tools for array manipulations, deep learning, data access, and visualization.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.layers import Dense, Reshape, UpSampling2D,
Conv2D, BatchNormalization, LeakyReLU, Flatten, Input
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
```

Step 2 : This function creates a "generator" model. Think of the generator as an artist that creates paintings (in this case, images). It starts with random doodles and, over time, tries to perfect its art to produce realistic images.

```
# Generator
def build_generator():
    model = Sequential()
    model.add(Dense(128 * 8 * 8, activation="relu", input_shape=(100,)))
    model.add(Reshape((8, 8, 128)))
    model.add(UpSampling2D())
    model.add(Conv2D(128, kernel_size=3, padding="same"))
    model.add(BatchNormalization())
    model.add(LeakyReLU(alpha=0.01))
    model.add(UpSampling2D())
    model.add(Conv2D(64, kernel_size=3, padding="same"))
    model.add(BatchNormalization())
    model.add(LeakyReLU(alpha=0.01))
    model.add(Conv2D(3, kernel_size=3, padding="same",
activation='tanh'))
    return model
```

Step 3: This function constructs a "discriminator" model. The discriminator acts like a critic. It looks at both the real and fake (produced by the generator) images and tries to distinguish between them.

```
# Discriminator
def build_discriminator():
    model = Sequential()
    model.add(Conv2D(32, kernel_size=3, strides=2, input_shape=(32, 32, 3), padding="same"))
    model.add(LeakyReLU(alpha=0.01))
    model.add(Conv2D(64, kernel_size=3, strides=2, padding="same"))
    model.add(BatchNormalization())
    model.add(LeakyReLU(alpha=0.01))
    model.add(Conv2D(128, kernel_size=3, strides=2, padding="same"))
    model.add(BatchNormalization())
    model.add(LeakyReLU(alpha=0.01))
    model.add(Flatten())
    model.add(Dense(1, activation='sigmoid'))
    return model
```

Step 4 : Here, we prepare our tools for training. We choose an optimizer, which helps in refining our models (think of it as a tool that helps the artist improve faster). Then, we set up a combined system where the generator creates an image, and the discriminator evaluates it.

```
# Compilation
optimizer = tf.keras.optimizers.legacy.Adam(0.0002, 0.5)
discriminator = build_discriminator()
discriminator.compile(loss='binary_crossentropy', optimizer=optimizer,
metrics=['accuracy'])
generator = build_generator()
z = Input(shape=(100,))
```

```
img = generator(z)
discriminator.trainable = False
validity = discriminator(img)
combined = Model(z, validity)
combined.compile(loss='binary_crossentropy', optimizer=optimizer)
```

Step 5: This function is the main training loop. It's like a repeated class between the artist (generator) and the critic (discriminator). The artist tries to make paintings, the critic evaluates them, and based on the feedback, the artist improves. This process is repeated many times (epochs). Periodically, the artist's work is displayed for us to see.

```
# Training function
def train(epochs, batch_size=128, save_interval=50):
    (x_train, _), (_, _) = cifar10.load_data()
    x_train = (x_train.astype(np.float32) - 127.5) / 127.5
    valid = np.ones((batch_size, 1))
    fake = np.zeros((batch_size, 1))
    for epoch in range(epochs):
        idx = np.random.randint(0, x_train.shape[0], batch_size)
        imgs = x_train[idx]
        noise = np.random.normal(0, 1, (batch_size, 100))
        gen_imgs = generator.predict(noise)
        d_loss_real = discriminator.train_on_batch(imgs, valid)
        d_loss_fake = discriminator.train_on_batch(gen_imgs, fake)
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)
        noise = np.random.normal(0, 1, (batch_size, 100))
        g_loss = combined.train_on_batch(noise, valid)
        if epoch % save_interval == 0:
            print(f"{epoch}/{epochs} [D loss: {d_loss[0]} | D accuracy: {100*d_loss[1]}] [G loss: {g_loss}]")
            save_imgs(epoch)
```

Here, real images (imgs) from the dataset are taken. The discriminator tries to identify them as real. It's like showing the detective real currency and telling him, "This is genuine."

The generator first creates some images (gen_imgs). These are then passed to the discriminator, which tries to identify them as fake. It's like the counterfeiter trying to pass off fake currency, and the detective tries to detect it.

The generator tries to improve itself by fooling the discriminator. It creates more images and aims for the discriminator to misclassify them as real. The feedback from the discriminator is used to refine the generator. It's like the counterfeiter learning from his mistakes whenever the detective catches the fake currency.

Step 6: This function showcases the generator's (artist's) work. After creating a set of images, they are displayed in a grid. This allows us to visually track how the artist is improving over time.

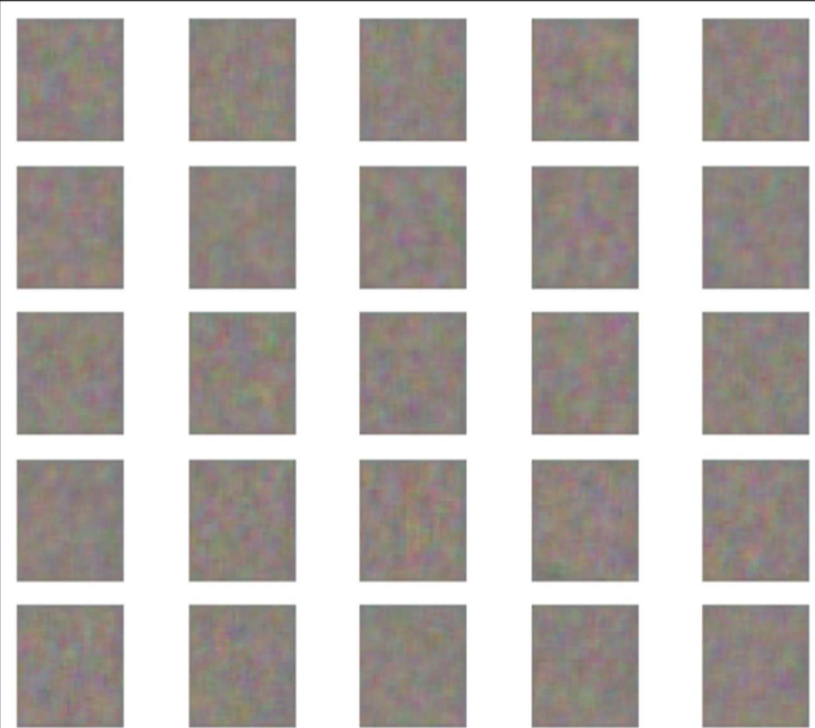
```
# Image saving function
def save_imgs(epoch):
    r, c = 5, 5
    noise = np.random.normal(0, 1, (r * c, 100))
    gen_imgs = generator.predict(noise)
    gen_imgs = 0.5 * gen_imgs + 0.5
    fig, axs = plt.subplots(r, c)
    cnt = 0
    for i in range(r):
        for j in range(c):
            axs[i,j].imshow(gen_imgs[cnt, :, :, :])
            axs[i,j].axis('off')
            cnt += 1
    plt.show()
```

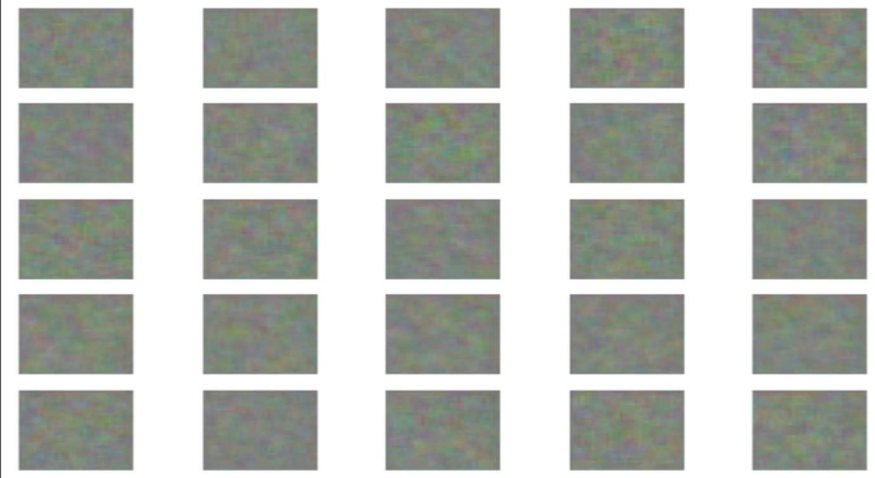
The `save_imgs` function lets us visualize how well the generator is doing. After certain intervals, the generated images are plotted, giving a visual idea of the generator's current performance. It's like displaying the latest counterfeits to see how close they are to real currency.

Step 7 : Finally, the training starts! We're conducting 25 classes (epochs) with the artist and critic. Every 5 classes, we'll display the artist's work to see how it's doing.

```
train(epochs=25, batch_size=32, save_interval=5)
```

```
1/1 [=====] - 0s 354ms/step  
0/25 [D loss: 0.8288569152355194 | D accuracy: 45.3125] [G loss: 0.7146297097206116]  
1/1 [=====] - 0s 169ms/step
```





```

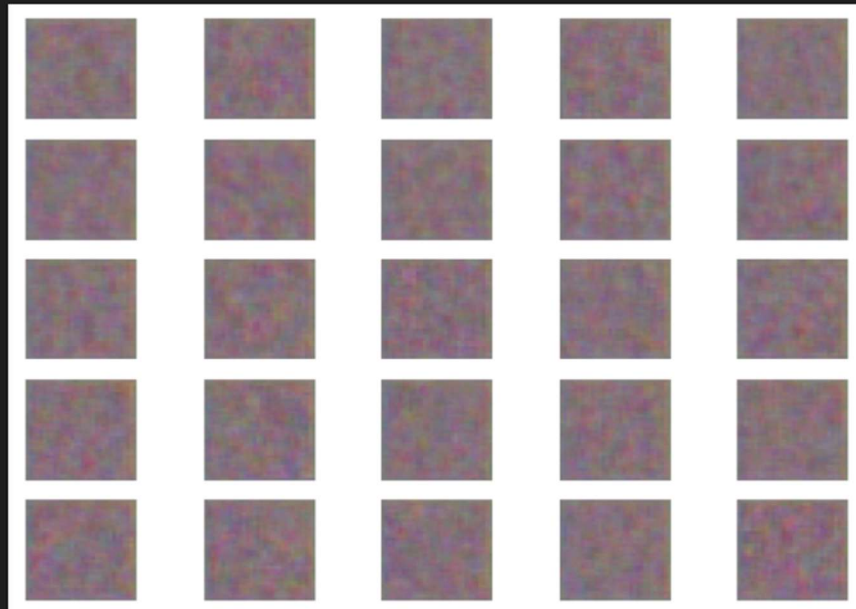
J\J [=====] - 0s 85ms/step
2\52 [D loss: 0.00328485831011 | D accuracy: 20.52] [e loss: 0.010000000000000001]
J\J [=====] - 0s 81ms/step
J\J [=====] - 0s 80ms/step
J\J [=====] - 0s 83ms/step
J\J [=====] - 0s 81ms/step
J\J [=====] - 0s 81ms/step

```

```

1/1 [=====] - 0s 80ms/step
1/1 [=====] - 0s 82ms/step
1/1 [=====] - 0s 89ms/step
1/1 [=====] - 0s 89ms/step
1/1 [=====] - 0s 71ms/step
15/25 [D loss: 0.41912737488746643 | D accuracy: 89.0625] [G loss: 0.680911123752594]
1/1 [=====] - 0s 119ms/step

```





In essence, as training progresses, the generator becomes better at creating images, and the discriminator becomes better at detecting fake images. The hope is that, by the end of training, the generator is so good that the discriminator (and even humans) will have a tough time telling its creations apart from real images.

Challenges in Implementation

Mode Collapse: It's a situation where the generator produces limited varieties of samples, or even the same sample, regardless of the input. Introduce some noise or randomness into the process or use different GAN architectures that are resistant to mode collapse.

Training Instability: GANs can sometimes result in the loss going to NaN or the generation of nonsensical images. Use techniques like gradient clipping, different loss functions, or architectures like WGAN.

Convergence Issues: Traditional GANs can sometimes not converge, resulting in poor image generation. Using architectures like WGAN or spectral normalization can help.

Conclusion

Generative Adversarial Networks, when trained on the CIFAR-10 dataset, can produce novel and relevant images that can be potential augmentations to the dataset. While GANs come with their own set of challenges, careful design choices and implementations can lead to very satisfactory results. This method opens the doors to endless possibilities in the realms of image generation, data augmentation, and beyond.